Duoc UC – Sede Maipú

Capstone 2026 – Asignatura APT122

Evento Cultural

Documentación Técnica del Proyecto

Plataforma chilena de eventos culturales

Integrante	Rol principal
Roberto Riffo	Líder de proyecto / Frontend / UX
José Miguel Moncada	Base de datos / QA
David Herrera	Backend e integraciones

Roles según el design brief aprobado en Fase 1 (*design_brief_capstone.docx*).

Stack del sistema: Python (scraper con conectores, NVIDIA NIM y checkpoints SQLite), Supabase/PostgreSQL con Row Level Security, Edge Functions (registro, cuenta, publicación, moderación y recuperación), Angular 22 + Ionic 9 (web y móvil, catálogo real y autenticación directa vía `@supabase/supabase-js`), Leaflet + OpenStreetMap (mapa interactivo con pines georreferenciados) y GitHub Actions para automatización.

Versión 1.4 — revisión documental

3 de noviembre de 2026

Copia revisada: original conservado sin modificaciones.
Contraste de fuentes locales, migraciones y registros históricos.
No certifica configuración remota ni funcionamiento actual de la aplicación.

Índice

1. Introducción y alcance **3**
- 1.1. Estructura del sistema . . . 3
- 1.2. Respuesta a la evaluación anterior . . . 3

2. Arquitectura general del sistema **3**
- 2.1. Alcance y fuente de verdad . . . 3
- 2.2. Contratos y límites de confianza . . . 4

3. Arquitectura del scraper **5**

4. Arquitectura del frontend **6**
- 4.1. Stack y organización . . . 6
- 4.2. Descubrimiento y personalización . . . 6
- 4.3. Cuenta y perfil . . . 6
- 4.4. Administración y publicación . . . 7
- 4.5. Despliegue y pendientes . . . 7

5. Modelo de datos (Supabase / PostgreSQL) **8**
- 5.1. Fuentes y alcance . . . 8
- 5.2. ER dividido por dominio . . . 9
- 5.3. Inventario y diccionario resumido . . . 10
- 5.4. Autorización de Fase 3 y recuperación . . . 11
- 5.5. Vista public_event_catalog, permisos y RLS . . . 11

6. Diagramas UML y flujos **13**
- 6.1. Casos de uso actuales . . . 13
- 6.2. Registro, sesión, publicación y moderación . . . 14
- 6.3. Clases y componentes . . . 14

7. Requisitos no funcionales **15**
- 7.1. RNF-01 — Interfaz clara e intuitiva . . . 15
- 7.2. RNF-02 — Diseño responsive / multiplataforma . . . 15
- 7.3. RNF-03 — Tiempos de carga aceptables . . . 16
- 7.4. RNF-04 — Protección de datos de usuarios y organizadores . . . 16
- 7.5. RNF-05 — Control de acceso por rol . . . 16
- 7.6. RNF-06 — Disponibilidad del servicio . . . 17
- 7.7. RNF-07 — Soporte de crecimiento de datos . . . 17
- 7.8. RNF-08 — Consistencia y normalización de la información . . . 17
- 7.9. RNF-09 — Código mantenible y documentado . . . 17
- 7.10. RNF-10 — Contenido en español y contexto chileno . . . 18
- 7.11. RNF-11 — Despliegue reproducible . . . 18
- 7.12. RNF-12 — Buenas prácticas de accesibilidad web . . . 18

8. Pruebas y evidencia **19**
- 8.1. Ejecuciones históricas, no regresión actual . . . 19
- 8.2. CI con referencias concretas . . . 19
- 8.3. Comandos de reproducción, no ejecutados en esta revisión . . . 19

9. Despliegue sin Docker y brecha de aceptación **20**

10. Innovación y límites del MVP **20**

11. Estándares de desarrollo y evidencia del avance **20**
11.1. Estándares, convenciones y herramientas 20
11.2. OAuth y alcance de la afirmación PKCE 21
11.3. Patrones de diseño y mecanismos observables 21
11.4. Roles planificados y aportes 22

12. Trazabilidad de la revisión **22**

13. Pendientes y evidencia faltante **23**

Referencias **25**

1. Introducción y alcance

Este documento consolida el estado técnico de Evento Cultural al 03-11-2026 a partir del código local actual, las funciones Edge y configuración de la aplicación. Conserva como línea base los requerimientos de Fase 1, el modelo de datos y la evidencia del scraper y QA de Fase 2. No certifica el esquema remoto ni reemplaza una ejecución de pruebas.

Convención de honestidad documental: este documento distingue explícitamente entre lo *implementado* (respaldado por código), lo *parcial* (respaldado solo en parte) y lo *pendiente* (información humana o trabajo futuro). Nada se afirma como terminado sin respaldo; los puntos abiertos se marcan con **Pendiente de completar por el equipo.**

1.1. Estructura del sistema

El sistema se compone de tres bloques independientes que se comunican a través de la base de datos: el **scraper** (recolección y enriquecimiento de datos), la **base de datos** (catálogo canónico en Supabase/PostgreSQL) y el **frontend** (aplicación web y móvil), con **GitHub Actions** como automatización de la ejecución del scraper (ver Figura 1 en la Sección 2).

1.2. Respuesta a la evaluación anterior

La planilla de Fase 2 registra 74/100 y nota 5,1. Los descuentos están en los indicadores 3 (15/25), 4 (3/5), 5 (3/5), 6 (12/20) y 8 (6/10). No contiene observaciones narrativas específicas. Se corrigen precisión técnica, estado del producto, fuentes y trazabilidad; la entrega en Git y el dominio oral del equipo requieren evidencia adicional. El documento *Respuesta a la evaluación.docx* vincula cada indicador con acciones sin prometer una nueva calificación. Código implementado no equivale a prueba aprobada.

2. Arquitectura general del sistema

2.1. Alcance y fuente de verdad

Esta revisión toma como fuente principal el código local de `EventoCultural_Integrado/EventoCultural_Integrado`. La rama de referencia es `https://github.com/sunsitox/Proyecto_Capston_Evento_Cultural/tree/deploy/vercel-preview`. No se certifica igualdad entre la copia local y esa rama ni que todas las funciones locales estén desplegadas. Se distingue implementación, configuración remota y verificación de extremo a extremo.

- **Recolección:** scraper Python independiente, con conectores, saneamiento, NVIDIA NIM selectivo y checkpoints SQLite. Su evidencia operativa conservada corresponde al ciclo del 11-10-2026; no constituye una nueva ejecución del scraper.
- **Persistencia:** Supabase/PostgreSQL, catálogo y perfiles con políticas RLS en migraciones locales; Supabase Auth para sesiones y Storage para avatares.
- **Cliente:** SPA Angular 22 e Ionic 9, con servicios TypeScript y signals. Consulta catálogo y autenticación mediante `@supabase/supabase-js`. El código Python de `backend/` es legado y no es una dependencia del flujo web actual.
- **Servidor:** Supabase Edge Functions para operaciones privilegiadas: `registrar-cuenta`, `admin-account`, `publicar-evento`, `admin-eventos`, `reset-password-request`, `reset-password-confirm` y `recomendaciones-semanal`. La clave de servicio pertenece exclusivamente al servidor.
- **Entrega:** Vercel para la SPA, GitHub Actions para el scraper y Capacitor para Android. No se utiliza Docker en la estrategia adoptada.

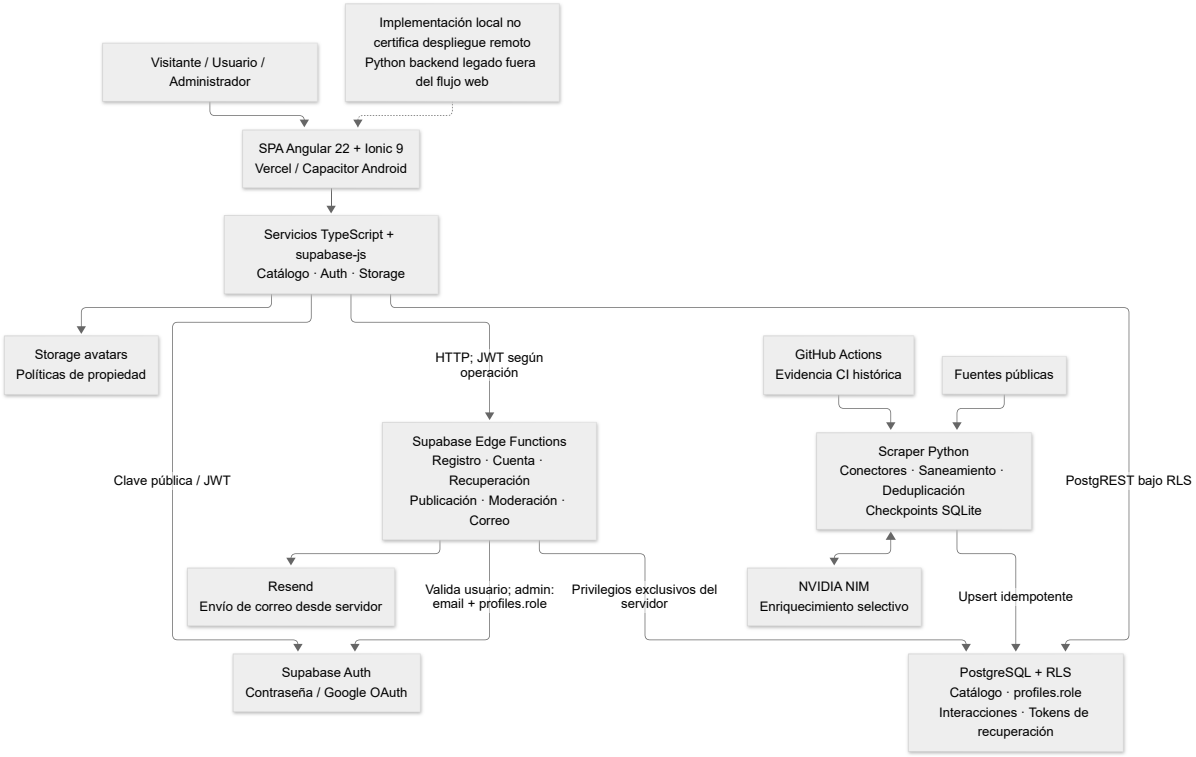


Figura 1: Arquitectura actual; fuente editable: Diagramas/01_arquitectura_general.mmd.

2.2. Contratos y límites de confianza

Origen y destino	Contrato	Regla de acceso
Cliente a Supabase	Catálogo mediante PostgREST; sesión mediante Auth	Clave pública y JWT del usuario; las políticas RLS gobiernan los datos.
Registro a función Edge	Correo, contraseña y nombre	Crea cuenta confirmada mediante Admin API; luego el cliente inicia sesión. No comprueba la propiedad del buzón.
Usuario a publicar-evento	Datos validados del evento y JWT	Identidad obtenida con getUser y user_metadata.rol=organizador (autodeclarado); crea draft y no acepta status/source_id del cliente. No prueba propiedad estable de organizador.
Administrador a admin-eventos	Listado y acciones de moderación	Valida JWT y profiles.role=admin; rechaza provider explícito distinto de email antes de utilizar privilegios de servicio.
Recuperación a funciones Edge	Solicitud por correo y confirmación con token	JWT firmado y registro password_reset_tokens con caducidad y marca de uso; requiere migración y secretos.
Servidor a correo	API de Resend	Clave privada en el entorno de funciones. Envío implementado no equivale a recepción demostrada.

3. Arquitectura del scraper

El pipeline carga fuentes declarativas, selecciona conectores por Strategy/Factory, sanea y deduplica eventos, normaliza ubicaciones, exporta JSON/CSV y sincroniza Supabase por upsert. Los checkpoints SQLite permiten reanudar trabajo y reutilizar resultados de IA de eventos sin cambios. El límite configurado de NVIDIA es 40 RPM; no se presenta como una garantía contractual del proveedor.

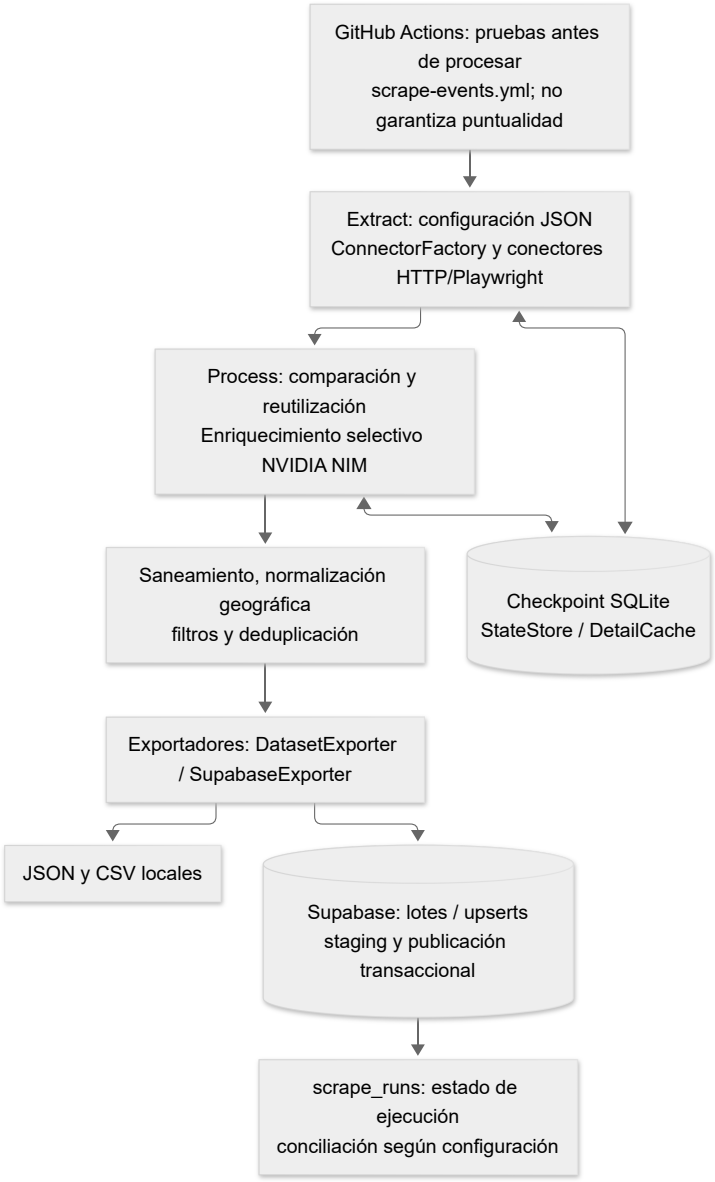


Figura 2: Pipeline resumido contrastado con EventPipeline y el workflow local; no acredita ejecución actual.

El flujo restaura checkpoints, extrae por conectores, compara cambios, enriquece selectivamente, sanea y deduplica, exporta y sincroniza mediante lotes/upserts; registra scrape_runs. La configuración del workflow solicita 40 RPM a NVIDIA y separa extract/process; no garantiza la cuota del proveedor.

El inventario de 21 fuentes permanentes y una estacional y las ejecuciones de CI corresponden a evidencia histórica. No se infiere que todas sigan disponibles hoy. Los conectores aíslan fallos por fuente; la conciliación y el borrado de vencidos dependen de la configuración y de una extracción

completa. GitHub Actions no garantiza puntualidad: el desfase del run 25 sigue registrado como DEF-CI-001.

4. Arquitectura del frontend

4.1. Stack y organización

Según `package.json`: Angular 22.0.1, Ionic 9, TypeScript 6, Capacitor 8, supabase-js 2, Leaflet 1.9.4, RxJS 7.8 y Vitest `^4.0.8` con jsdom `^26.0.0` (rangos declarados, no versiones de ejecución). `core/` concentra modelos y servicios; `pages/` las pantallas; `features/home/` los layouts; y `layout/` la navegación adaptativa. Las versiones resueltas deben reproducirse con `package-lock.json`.

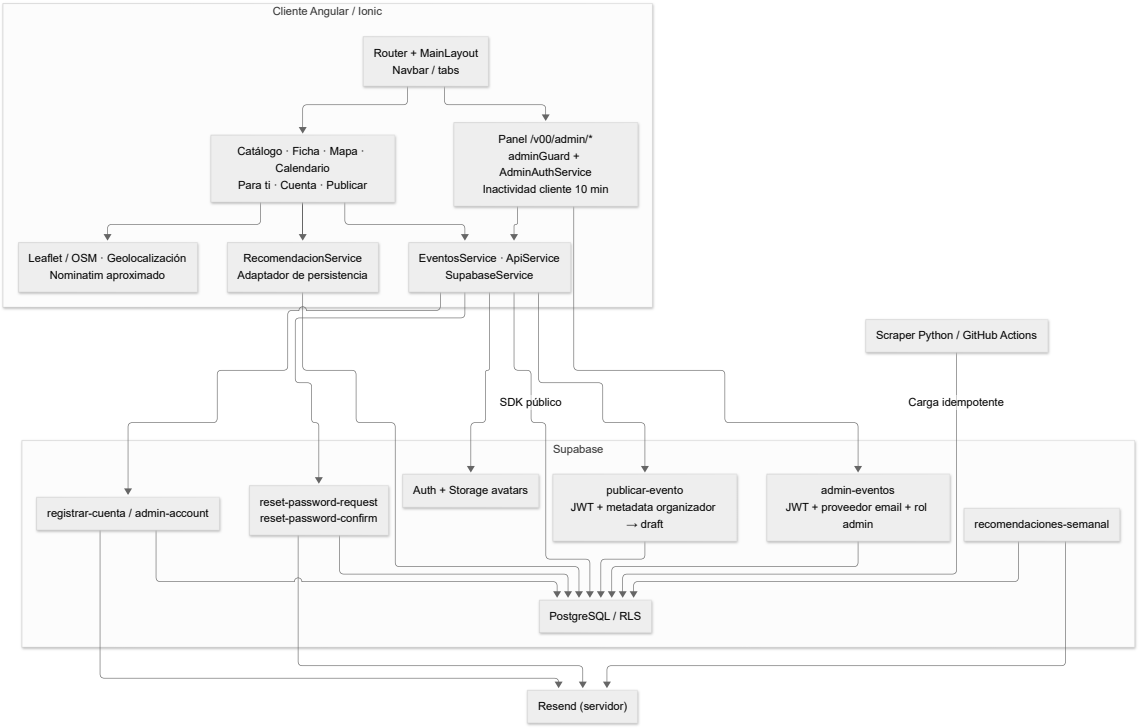


Figura 3: Componentes actuales y separación cliente/servidor.

4.2. Descubrimiento y personalización

El catálogo publicado alimenta búsqueda, filtros combinados, detalle, calendario y mapa Leaflet/OpenStreetMap. Nominatim aporta geocodificación de respaldo con cola, caché y límites; los resultados aproximados no sustituyen coordenadas verificadas. La geolocalización usa Capacitor en Android y alternativa web. Las recomendaciones se calculan por señales de interés, no por un modelo generativo: `RecomendacionService` calcula afinidad y razones; el adaptador de persistencia almacena interacciones en `user_event_interactions` y preferencias en metadata. Los invitados mantienen estado en memoria. El respaldo de eventos de demostración no debe confundirse con una consulta remota exitosa.

4.3. Cuenta y perfil

El registro principal llama `registrar-cuenta`, que crea el usuario con `email_confirm=true`; el cliente ejecuta `signInWithPassword` sin exigir verificación del buzón. La función intenta enviar un correo de registro exitoso por Resend. Existe respaldo legacy con `signUp`: si la función no está

disponible, puede reaparecer la confirmación de Supabase. Por tanto, la experiencia solicitada requiere desplegar y configurar la función, no solo cambiar el formulario.

El perfil permite editar nombre, contraseña e imagen. Storage utiliza el bucket `avatars`; el metadata `avatar_url_custom` tiene prioridad sobre la imagen OAuth, y esta se utiliza si no hay imagen propia. La identidad reactiva alimenta perfil y navegación. Google OAuth tiene botón y callback implementados; su configuración remota y la recepción de correos no se certifican por inspección de código. La recuperación actual usa las funciones `reset-password-request` y `reset-password-confirm` y `/restablecer-contrasena?token=...`, no únicamente el flujo antiguo de GoTrue.

4.4. Administración y publicación

Las rutas `/v00/admin/*` requieren `adminGuard`; no enlazarlas públicamente no constituye una medida de autorización. `AdminAuthService` consulta `profiles.role` y solo admite `admin`, con proveedor email: visitor, editor y Google son rechazados. El rol editable de metadata no es autoridad administrativa. La función `admin-eventos` valida getUser y profiles.role en servidor. Su condición rechaza app_metadata.provider cuando existe como cadena y difiere de email; no se declara una prueba ejecutada para identidades sin ese campo. El cliente cierra la sesión tras 10 minutos sin interacción, comprobados cada 15 segundos, y redirige al ingreso con aviso. La última actividad se reinicia al recargar; no es un timeout absoluto persistido ni una revocación impuesta por el servidor.

La publicación autenticada exige user_metadata.rol=organizador, atributo de autoservicio, y vincula organizer_id por nombre, no por una FK estable entre usuario y organizador. No se equipara ese atributo con el rol administrativo. Crea un borrador mediante `publicar-evento`; `admin-eventos` implementa listado administrativo, publicación y supresión. El dashboard calcula métricas del catálogo cargado en el cliente: no equivale a analítica comercial ni a estadísticas certificadas de uso. La ocultación de fuentes en el navegador no equivale a detener el scraper ni a persistir una decisión global.

4.5. Despliegue y pendientes

`angular.json` genera `www`; `vercel.json` ya incluye la reescritura SPA a `/index.html`. Existe proyecto Android y APK debug local; esto no demuestra instalación, GPS u OAuth en dispositivo físico ni publicación en tienda. Quedan por registrar: regresión del ciclo QA histórico, pruebas remotas de autorización, correo y recuperación, aislamiento entre usuarios, pruebas nativas y mediciones de rendimiento/accesibilidad. Ninguno de esos resultados se deduce de una compilación.

5. Modelo de datos (Supabase / PostgreSQL)

5.1. Fuentes y alcance

El modelo revisado se obtiene de las migraciones SQL locales, no de una exportación remota. Las seis migraciones del scraper se encuentran en `tmp_scraper_repo/supabase/migrations/`: catálogo inicial (31-08), ubicación (02-09), enriquecimiento y controles de organizadores (11-09), estado de reescritura (12-09), publicación atómica (21-09) e interacciones (24-09), todas de 2026. La aplicación añade `20261024000100_password_reset_tokens.sql` y `20261024000200_avatars_bucket.sql` en su propio directorio `supabase/migrations/`. Son conjuntos de componentes diferentes. El DBML y diccionario base están en `tmp_scraper_repo/docs/evento_cultural_modelo.dbml` y `tmp_scraper_repo/docs/DATABASE.md` (03-10-2026). Se contrastaron con SQL; no incluyen password_reset_tokens del frontend. La copia integrada editable `Diagramas/modelo_datos.dbml` añade esa tabla sin modificar las fuentes originales. Se corrige la afirmación general de que no existían migraciones locales: pueden haber faltado en un entregable antiguo, pero están disponibles en las copias aquí revisadas. No se comprobó cuáles se aplicaron a Supabase ni qué cambios manuales posteriores existen.

5.2. ER dividido por dominio

Para legibilidad se muestran claves y relaciones principales, no cada columna. El inventario y diccionario resumido siguientes completan la lectura. `auth_users` es exclusivamente el alias conceptual de Mermaid para la tabla real `auth.users`, gestionada por Supabase Auth; no es una tabla adicional en public ni una copia de credenciales.

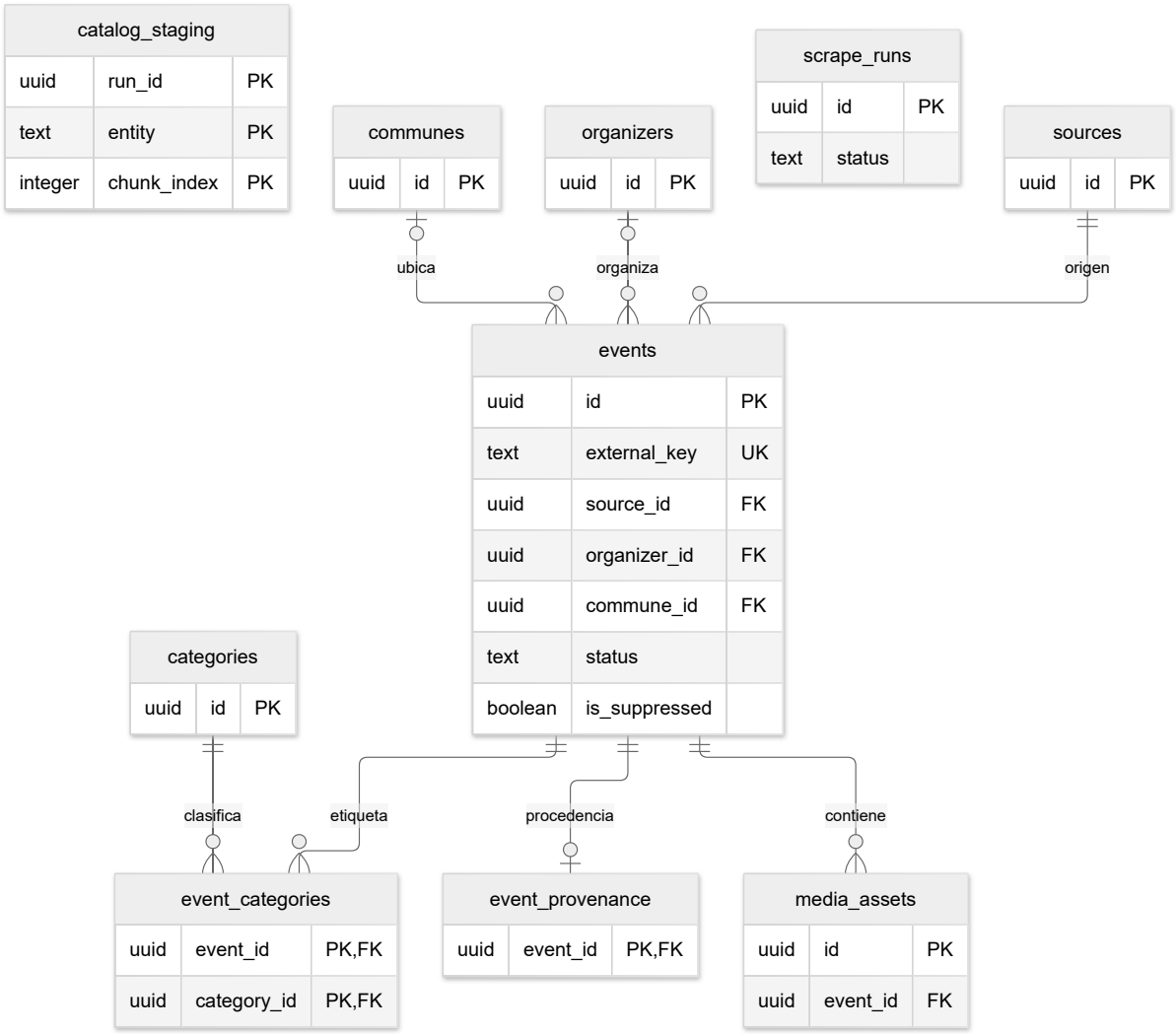


Figura 4: ER del catálogo local: claves, opcionalidad y relaciones principales.

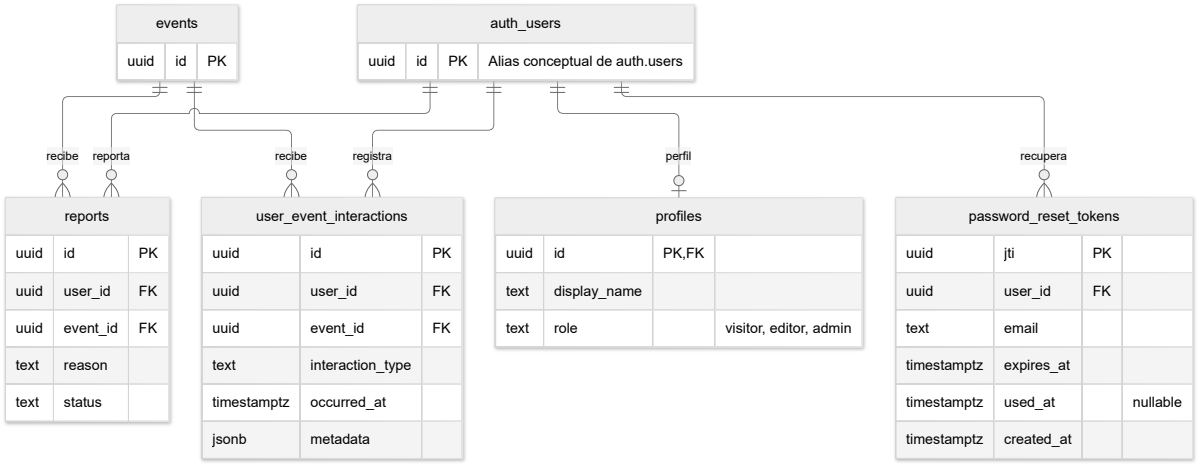


Figura 5: Identidad, perfiles, interacciones y recuperación; auth_users representa auth.users.

5.3. Inventario y diccionario resumido

Todas las tablas siguientes son del esquema public salvo auth.users y los objetos de Storage. No se fija un número de columnas de events como propiedad del esquema remoto.

Cuadro 2: Entidades y claves contrastadas con SQL local.

Entidad	Claves, finalidad y alcance
sources	id UUID PK, name único sin distinguir mayúsculas; fuente, website_url, is_active y last_seen_at.
organizers	id UUID PK, name único; entity_type e is_blocked, blocked_reason, blocked_at. La publicación busca por nombre: no hay FK demostrada a usuario propietario.
communes	id UUID PK; name, region y country, identidad territorial normalizada.
categories	id UUID PK, name único; event_categories une evento/categoría con PK compuesta y dos FK.
events	id UUID PK; external_key única; source_id obligatoria; organizer_id y commune_id opcionales. Datos de ficha, fechas, location, location_precision/source, coordenadas, status, is_suppressed, marcas de extracción y actualización. El ER resume claves, no el diccionario completo de columnas.
media_assets	id UUID PK, event_id FK, url, media_type, is_primary; unicidad por evento/URL.
`event_provenance`	event_id PK/FK a events; original_url, source_description, ocr_text, is_rewritten y updated_at; la migración del 12-09 expone ese estado en match_existing_events, no añade otra columna; información privada de procedencia.
scrape_runs	id UUID PK; started_at, finished_at, status (running/succeeded/failed), events_count, error_message y metadata. Sin FK a events en el SQL revisado.

Entidad	Claves, finalidad y alcance
catalog_staging	PK (run_id, entity, chunk_index), payload JSONB y created_at; no se dibuja una FK de run_id a scrape_runs porque el SQL no la declara. publish_staged_catalog aplica la carga transaccional.
auth.users	Tabla externa administrada por Auth; id UUID referenciada por los datos de cuenta. auth_users es solo su nombre conceptual en el dibujo.
profiles	id PK/FK a auth.users; display_name, role (visitor/editor/admin) y marcas de tiempo. La migración inicial concede actualizar display_name, no role, a authenticated.
`user_event_interactions`	id PK, user_id FK a auth.users y event_id FK a events; interaction_type, occurred_at, metadata JSONB. La migración del 24-09 migra favorites a interested y elimina favorites.
reports	id PK, user_id y event_id FK; reason, status (pending/reviewed/resolved) y marcas de tiempo. Tabla disponible no acredita un flujo ciudadano completo.
`password_reset_tokens`	jti UUID PK; user_id UUID FK a auth.users con ON DELETE CASCADE; email TEXT NOT NULL; expires_at TIMESTAMPTZ NOT NULL; used_at TIMESTAMPTZ nullable; created_at TIMESTAMPTZ NOT NULL DEFAULT now(). Índices por email y expires_at.
Storage avatars	No es una tabla de catálogo: bucket en storage.buckets y objetos en storage.objects. Migración local propone lectura pública y escrituras de authenticated bajo carpeta auth.uid(). No acredita políticas aplicadas remotamente.

Los UUID v5 deterministas son una decisión del exportador del scraper, no una restricción global del tipo UUID en PostgreSQL. La función publicar-evento utiliza crypto.randomUUID() y claves externas org:usuario:tiempo:sufijo; no se afirma que toda alta tenga UUID v5.

5.4. Autorización de Fase 3 y recuperación

La administración usa profiles.role y una comprobación de proveedor en admin-eventos, además del guard cliente. La publicación usa user_metadata.rol=organizador y crea status=draft. Ese metadata de autoservicio no demuestra propiedad administrativa ni relación estable usuario–organizador; tampoco debe confundirse con profiles.role.

password_reset_tokens registra el jti del token temporal emitido por reset-password-request. reset-password-confirm valida token, vencimiento y marca de uso antes de completar el cambio. La migración habilita RLS sin crear políticas, pero no contiene un REVOKE/GRANT explícito; no basta para describir todos los privilegios efectivos remotos. Tampoco se certifica consumo atómico ante concurrencia ni recuperación extremo a extremo mediante inspección estática. La migración del bucket avatars es separada de la recuperación y del scraper.

5.5. Vista public_event_catalog, permisos y RLS

`20260911000100_content_enrichment_and_organizer_controls.sql` declara:

```
create view public.public_event_catalog
with (security_invoker = true) as
select e.*, o.name as organizer_name, s.name as source_name
from public.events e
join public.sources s on s.id = e.source_id
left join public.organizers o on o.id = e.organizer_id
where e.status = 'published' and not e.is_suppressed;
revoke all on public.public_event_catalog from public;
grant select on public.public_event_catalog
  to anon, authenticated, service_role;
```

La misma migración define RLS en events para status published, no suprimido y `not public.is_organizer_blocked(organizer_id)`; modifica las políticas de organizers, event_categories y media_assets. La función is_organizer_blocked es SECURITY DEFINER con concesión EXECUTE a anon/authenticated/service_role. La migración inicial concede SELECT en tablas base y la del 11-09 restringe la lectura de organizers a columnas identificadas. La política de sources exige is_active.

No existe herencia automática de RLS por el mero hecho de crear una vista. Según CREATE VIEW de PostgreSQL (referencias), security_invoker=true aplica los privilegios y políticas del invocador sobre las relaciones subyacentes; por defecto se usan los del propietario. Aquí se acredita el mecanismo en SQL local, no su despliegue. service_role puede eludir RLS y no equivale al usuario público. Deben verificarse versión compatible (PostgreSQL 15 o posterior), propietario, reloptions, definición efectiva, permisos base/funciones y pg_policies; además de pruebas con identidades separadas. No se consultó el esquema remoto. No se concluye vulnerabilidad ni conformidad por esa limitación.

6. Diagramas UML y flujos

6.1. Casos de uso actuales

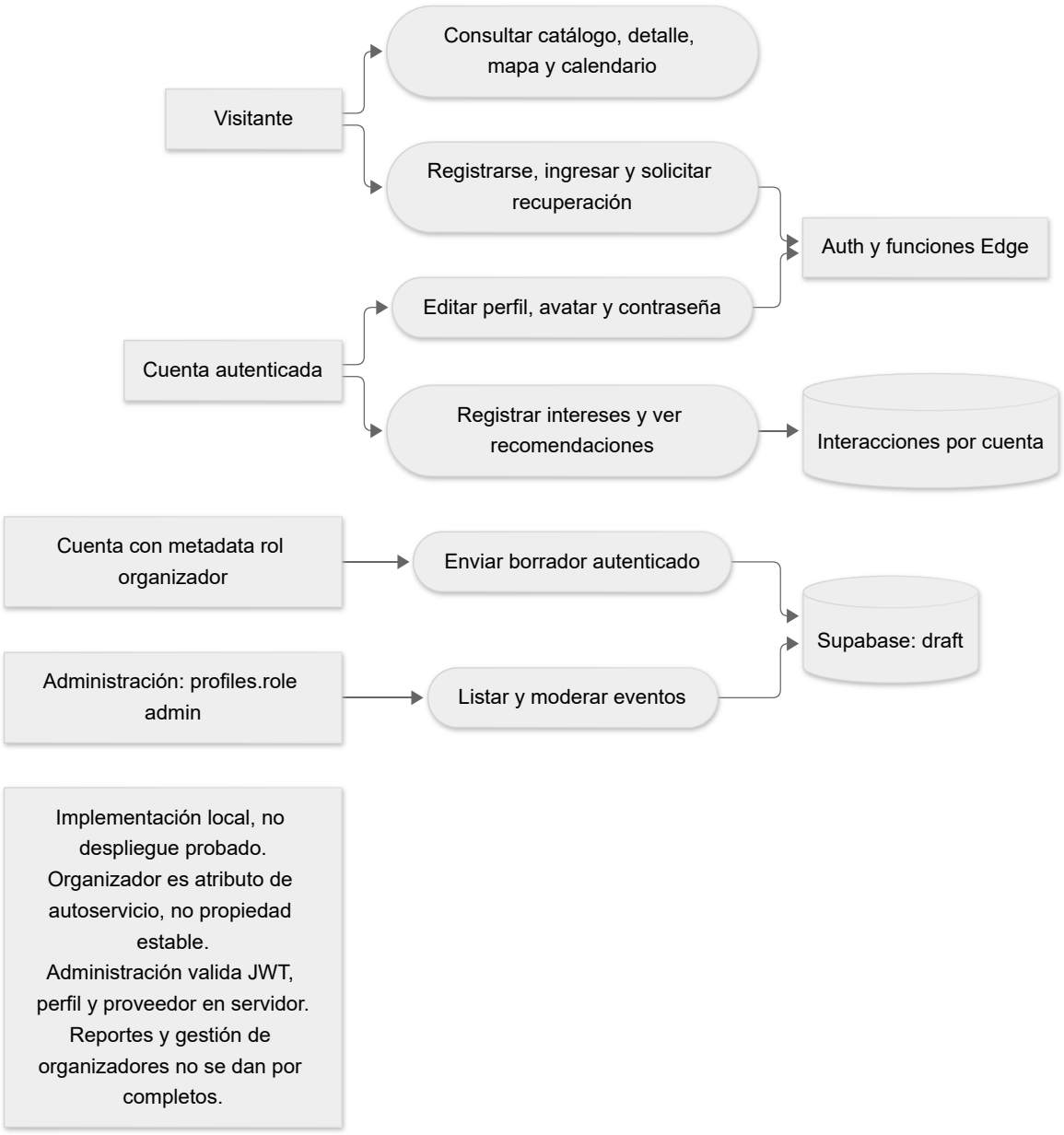


Figura 6: Casos de uso locales; conceptual, no certificación de despliegue.

El visitante descubre eventos y accede al registro; el usuario autenticado gestiona cuenta, perfil e intereses. La condición de organizador es metadata autodeclarada para enviar borradores. El administrador usa su rol en profiles para moderar. Correo y OAuth tienen integración local; recepción, configuración y recorridos completos no se dan por probados. GitHub Actions y NVIDIA son actores técnicos del scraper, representados en su pipeline, no roles ciudadanos. Reportes y administración de organizadores no se presentan como completos.

6.2. Registro, sesión, publicación y moderación

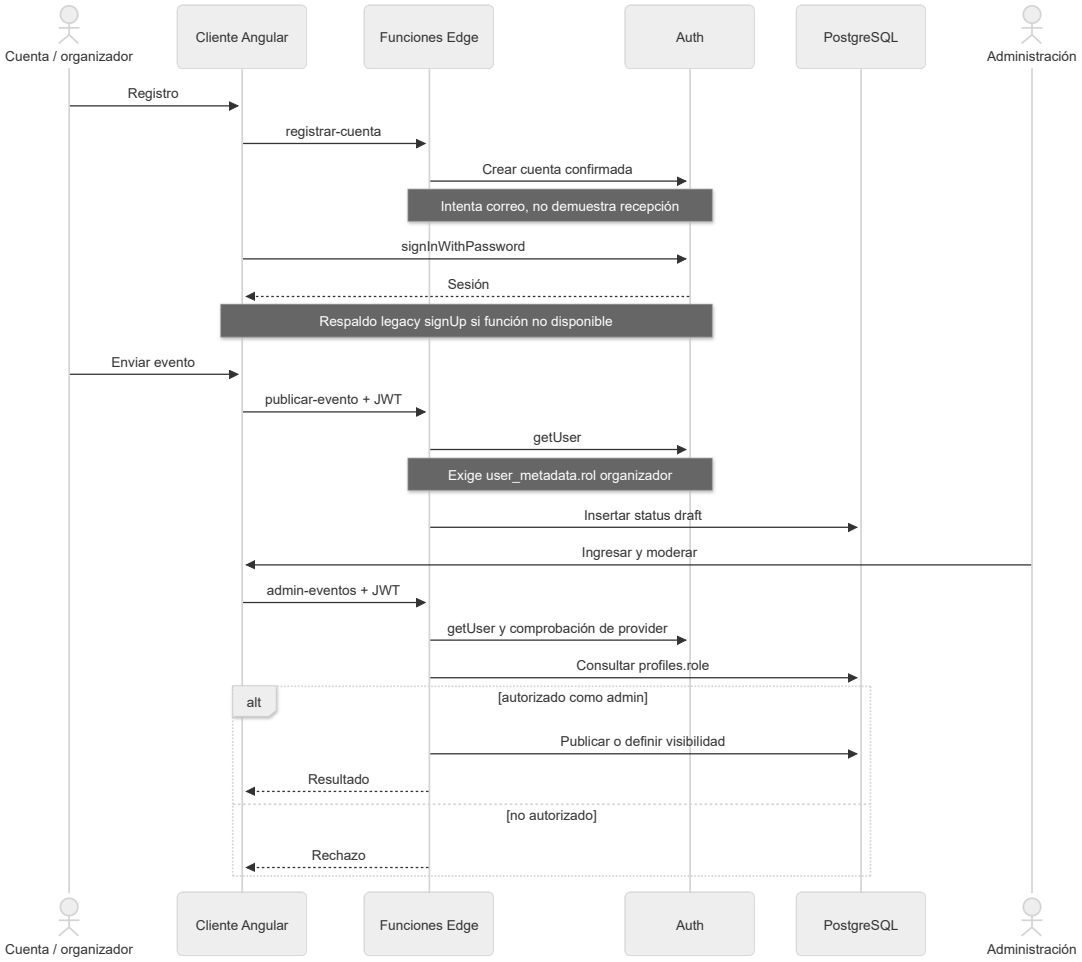


Figura 7: Secuencia resumida de cuenta y moderación.

El cliente solicita registrar-cuenta; la función crea una cuenta confirmada mediante Admin API e intenta enviar bienvenida. Después el cliente inicia sesión con contraseña. La confirmación de cuenta no prueba propiedad del buzón ni recepción del correo. Existe respaldo legacy signUp si falla la disponibilidad de la función. Para publicar, la función valida JWT y metadata de organizador antes de insertar draft. Para moderar, admin-eventos valida identidad, proveedor y profiles.role antes de publicar o cambiar is_suppressed. La secuencia representa el flujo nominal y el rechazo administrativo, no una ejecución registrada. La explicación de extracción y enriquecimiento está en la Sección 3.

6.3. Clases y componentes

El diagrama de componentes aparece en la Figura 3. Las clases siguientes proceden del núcleo local del scraper; las relaciones son dependencias conceptuales, no atributos o métodos traducidos inventados. Se corrigen PipelineState y NvidiaService por los nombres observados StateStore, DetailCache y NvidiaContentEnricher.

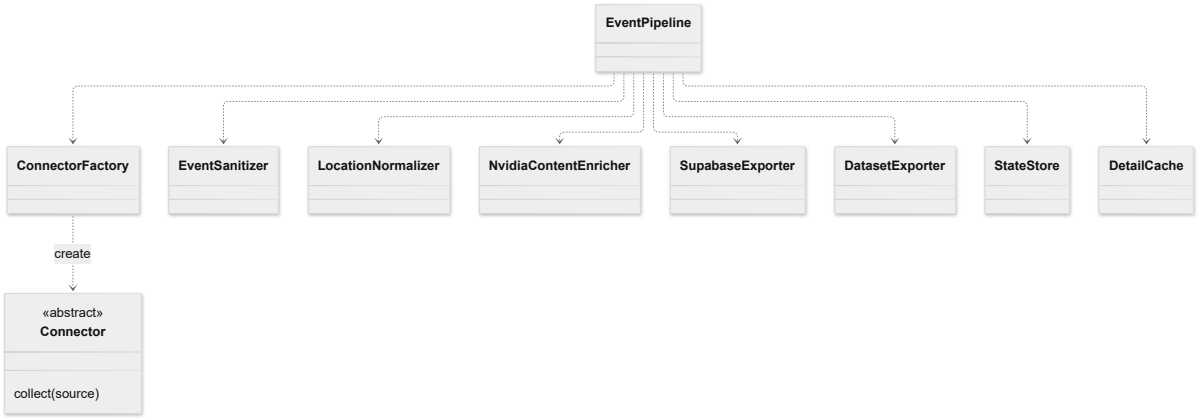


Figura 8: Clases locales y dependencias principales del scraper.

Los fuentes Mermaid se incluyen en Diagramas/. El diagrama de casos de uso es conceptual: si se exige notación UML estricta debe ratificarse la notación con el docente; no se presenta esa ratificación como ya realizada.

7. Requisitos no funcionales

Se mantienen RNF-01 a RNF-12 y sus enunciados/criterios de la matriz MoSCoW de Fase 3. Los criterios siguientes son **transcritos de esa matriz**, no umbrales nuevos ni resultados medidos. Su ratificación por el equipo/docente no consta en esta revisión. Las brechas de definición se señalan explícitamente; no se agregan propuestas de umbral.

7.1. RNF-01 — Interfaz clara e intuitiva

Categoría / prioridad: Usabilidad / Must have.

Enunciado: La plataforma debe tener una interfaz simple y comprensible tanto para ciudadanos como para organizadores sin conocimientos técnicos.

Criterio de aceptación existente: Se ejecuta una prueba de usabilidad con usuarios representativos y se registran hallazgos, severidad y acciones de mejora. La muestra y umbral de aprobación deben acordarse con el equipo.

Método y límite: Prueba con usuarios representativos y registro de tareas/hallazgos. Muestra y umbral pendientes de definición.

Evidencia local / estado registrado: Pendiente de validación. Acordar muestra/umbral y registrar prueba de usabilidad con usuarios representativos.

Referencia de implementación: Documentacion Metodologica/

7.2. RNF-02 — Diseño responsive / multiplataforma

Categoría / prioridad: Compatibilidad / Must have.

Enunciado: La interfaz web debe adaptarse correctamente a distintos tamaños de pantalla y funcionar de forma consistente entre la versión web y la móvil.

Criterio de aceptación existente: Las vistas principales se renderizan correctamente de 320 px a escritorio; navegación de tab bar en móvil y barra superior en escritorio.

Método y límite: Recorrido responsive de vistas principales a 320 px y escritorio, más matriz de navegadores y dispositivos. No ejecutado en esta revisión.

Evidencia local / estado registrado: Parcial. Diseño responsive y navegación dual implementados. Falta matriz de navegadores/dispositivos y prueba de 320 px a escritorio.

Referencia de implementación: src/app/layout/main-layout/; android/

7.3. RNF-03 — Tiempos de carga aceptables

Categoría / prioridad: Rendimiento / Should have.

Enunciado: Las búsquedas y listados de eventos deben responder en tiempos razonables aun cuando la base de datos crezca.

Criterio de aceptación existente: Con un catálogo de referencia de 1.000 eventos publicados, las consultas paginadas de listado y filtros deben medir p95 menor o igual a 2 segundos. La metodología y el entorno de prueba deben quedar registrados.

Método y límite: Medición de consultas paginadas y filtros sobre catálogo de referencia; registrar entorno, volumen y distribución p95. No hay medición de aceptación actual.

Evidencia local / estado registrado: Pendiente de medición. Paginación visual no equivale a paginación de consultas. Medir p95 $\leq$2 s con 1.000 eventos, entorno y metodología; no afirmar mejora sin medición.

Referencia de implementación: src/app/core/services/eventos.service.ts; src/app/pages/calendario/calendario.page.ts

7.4. RNF-04 — Protección de datos de usuarios y organizadores

Categoría / prioridad: Seguridad / Must have.

Enunciado: Las credenciales y datos personales de los usuarios deben almacenarse y transmitirse de forma segura.

Criterio de aceptación existente: Las credenciales no se almacenan en el código; el transporte usa HTTPS y el acceso a datos respeta las políticas de fila.

Método y límite: Inspección de cliente/build y HTTPS; pruebas de aislamiento RLS/Storage, registro y recuperación con cuentas distintas. Configuración remota no inspeccionada.

Evidencia local / estado registrado: Parcial. Auth y funciones Edge presentes; panel sin credenciales demo. Verificar HTTPS, secretos fuera del cliente, aislamiento RLS/Storage y abuso del registro/reset. Registro sin verificación no acredita propiedad del buzón.

Referencia de implementación: src/app/core/services/supabase.service.ts; supabase/functions/; supabase/migrations/20261024000100_password_reset_tokens.sql

7.5. RNF-05 — Control de acceso por rol

Categoría / prioridad: Seguridad / Must have.

Enunciado: El sistema debe distinguir permisos entre visitante, usuario registrado y administrador. El rol de organizador requiere un modelo de propiedad explícito antes de habilitar publicación o edición propia.

Criterio de aceptación existente: Las acciones protegidas se validan por rol y por propiedad. No se declara control por organizador hasta implementar la relación entre cuentas y organizadores.

Método y límite: Pruebas negativas sin sesión, visitor/editor/OAuth, cambio de rol y acceso directo a funciones; verificar propiedad. Pendiente ejecución registrada.

Evidencia local / estado registrado: Parcial. El guard exige profiles.role=admin y proveedor email; rechaza visitor, editor y Google. En servidor, admin-eventos valida getUser y profiles.role, y rechaza app_metadata.provider cuando existe como cadena y difiere de email; no hay prueba ejecutada para identidades sin ese campo. Publicación aún usa user_metadata.rol de autoservicio: definir alcance de organizador y propiedad estable. Faltan pruebas negativas y aislamiento.

Referencia de implementación: src/app/core/services/admin-auth.service.ts; src/app/pages/admin/admin.guard.ts; supabase/functions/admin-eventos/index.ts; supabase/functions/publicar-evento/index.ts

7.6. RNF-06 — Disponibilidad del servicio

Categoría / prioridad: Disponibilidad / Should have.

Enunciado: El servicio debe mantenerse disponible de forma estable con la infraestructura definida: base de datos administrada en Supabase, web desplegada en Vercel y ejecuciones automatizadas en GitHub Actions.

Criterio de aceptación existente: La web responde en el hosting definido y las ejecuciones automáticas se registran en scrape_runs con su estado.

Método y límite: Sondeos fechados de web y registros scrape_runs/CI; ventana y umbral de disponibilidad no definidos. CI histórico no constituye SLA.

Evidencia local / estado registrado: Parcial. Documentación registra runs #18–#25 en verde y DEF-CI-001 por horario fuera de ventana esperada. Verificar hosting vigente y monitoreo. Sin comprobación remota en esta revisión.

Referencia de implementación: Documentacion Tecnica/secciones/03-rnf-pruebas-docker-innovacion.tex

7.7. RNF-07 — Soporte de crecimiento de datos

Categoría / prioridad: Escalabilidad / Should have.

Enunciado: El modelo de datos y la arquitectura deben poder soportar un crecimiento sostenido en el número de eventos y organizadores sin degradar el servicio.

Criterio de aceptación existente: Al crecer el catálogo, el pipeline pagina y sincroniza por lotes sin cargar respuestas completas en memoria; re-ejecutar no duplica registros.

Método y límite: Carga incremental con medición de memoria y reejecución idempotente. Volumen, tolerancia y degradación admisible pendientes de definición.

Evidencia local / estado registrado: Parcial. Diseño por lotes/upserts documentado; falta prueba de crecimiento, memoria y reejecución sin duplicados. Diseño no equivale a prueba de carga.

Referencia de implementación: `tmp_scraper_repo/`; `src/app/core/services/eventos.service.ts`

7.8. RNF-08 — Consistencia y normalización de la información

Categoría / prioridad: Calidad de datos / Must have.

Enunciado: La información proveniente de distintas fuentes públicas debe normalizarse en fechas, ubicaciones y categorías para evitar inconsistencias o duplicados.

Criterio de aceptación existente: Eventos de distintas fuentes comparten fechas normalizadas, ubicación canónica y categorías; no se publican duplicados por URL original o clave compuesta.

Método y límite: Muestras de distintas fuentes y comparación por URL/clave, fechas y ubicación. Muestra y tolerancia de defectos pendientes de definición.

Evidencia local / estado registrado: Completado (implementación). Normalización/deduplicación implementadas; validar muestras actuales de fuentes y publicaciones de organizadores.

Referencia de implementación: tmp_scraper_repo/; Documentacion Tecnica/secciones/02-datos-uml.tex

7.9. RNF-09 — Código mantenible y documentado

Categoría / prioridad: Mantenibilidad / Should have.

Enunciado: El proyecto debe seguir buenas prácticas de desarrollo y mantener documentación mínima que permita a los tres integrantes del equipo dar mantención al sistema.

Criterio de aceptación existente: El repositorio mantiene documentación actualizada y las pruebas se ejecutan con un comando estándar.

Método y límite: Reproducción documental y comandos estándar en entorno identificado. Logs históricos disponibles; no prueban aptitud de mantenimiento por los tres integrantes.

Evidencia local / estado registrado: Parcial. Documentación Fase 3 revisada al 31-10 y PDF regenerados. Antecedente 23/23 del 27-10 no reejecutado; ejecutar controles afectados y registrar commit de entrega autorizado.

Referencia de implementación: package.json; Documentacion Tecnica/Documento_Tecnico_Capstone.pdf; Documentacion Metodologica/; Respuesta a la evaluación.docx

7.10. RNF-10 — Contenido en español y contexto chileno

Categoría / prioridad: Localización / Must have.

Enunciado: La plataforma debe estar en español y considerar formatos de fecha, dirección y división administrativa (comuna/región) propios de Chile.

Criterio de aceptación existente: Fechas, direcciones y divisiones administrativas siguen el formato chileno; la interfaz está en español.

Método y límite: Revisión de textos, acentos, fechas es-CL, comunas/regiones y zonas horarias. Muestra editorial de aceptación pendiente.

Evidencia local / estado registrado: Completado (implementación). UI en español y formato es-CL; revalidar acentos, divisiones administrativas y zona horaria de publicaciones.

Referencia de implementación: src/app/pages/calendario/calendario.page.ts; src/app/core/services/geocodificacion.service.ts

7.11. RNF-11 — Despliegue reproducible

Categoría / prioridad: Portabilidad / Must have.

Enunciado: El proyecto debe poder desplegarse de forma reproducible usando el stack definido (GitHub, GitHub Actions, Vercel, Capacitor/Android Studio), facilitando la integración continua entre los integrantes.

Criterio de aceptación existente: El repositorio debe incluir Dockerfile(s), docker-compose, variables de entorno documentadas e instrucciones reproducibles de levantamiento para los componentes aplicables.

Método y límite: Instalación y despliegue limpio según manual, y contraste con criterio Docker. Excepción docente no documentada; no cambiar el criterio para declarar cumplimiento.

Evidencia local / estado registrado: Parcial (decisión docente pendiente). Criterio original exige Dockerfile y compose, ausentes por decisión del equipo. No se elimina para simular cumplimiento: acordar excepción docente o implementar artefactos; comprobar despliegue reproducible.

Referencia de implementación: package.json; capacitor.config.ts; supabase/functions/; supabase/migrations/

7.12. RNF-12 — Buenas prácticas de accesibilidad web

Categoría / prioridad: Accesibilidad / Could have.

Enunciado: La interfaz debería seguir criterios básicos de accesibilidad (contraste, tamaños de fuente, navegación por teclado) para ampliar su alcance a más usuarios.

Criterio de aceptación existente: Auditoría WCAG 2.2 AA en vistas principales con contraste medido, teclado, foco visible y lector de pantalla. Errores con alert y éxitos con status; significado no dependiente solo del color; cierre etiquetado y tiempo suficiente para leer avisos.

Método y límite: Auditoría automática y revisión manual de teclado, foco, contraste y lector. axe histórico contiene hallazgos; no se declara WCAG AA aprobada.

Evidencia local / estado registrado: Parcial. Avisos con alert/status, fondo sólido, iconos y cierre accesible. Falta auditoría WCAG 2.2 AA, teclado y lector con contraste medido. Publicar redirige a los 1,8 s: no afirmar persistencia universal de avisos.

Referencia de implementación: src/app/pages/login/login.page.html; src/app/pages/publicar-evento/publicar-evento.page.ts; src/app/pages/restablecer-contrasena/restablecer-contrasena.page.ts

8. Pruebas y evidencia

8.1. Ejecuciones históricas, no regresión actual

En `DocumentacionMetodologica/Evidencias_QA_Fase3_2026-10-31/`, controles.json registra comandos, directorios, inicio y exit_code. frontend_tests.log registra 25 aprobadas (4 archivos) y scraper_tests.log 140 con OK; frontend_build.log y frontend_lint.log tienen código 0 según controles.json. Los archivos axe_home.json y axe_login.json conservan hallazgos de accesibilidad. Son resultados del 31-10-2026, no repetidos aquí. Los inicios de controles.json no incluyen zona horaria; el log frontend incluye 2026-10-31T08:20:45.391Z. No se inventa un SHA para esa ejecución.

El ciclo del 11-10-2026, `Evidencias_QA_Frontend_2026-10-11/README.md`, registra 13 escenarios (6 aprobados, 6 observados, 1 fallido) y 13 defectos; su evidencia es histórica. No se cierran por esta revisión. El antecedente 23/23 del 27-10 se conserva como declaración previa: no se usa como aceptación de la versión actual.

8.2. CI con referencias concretas

La carpeta `Evidencias_CI_Scraper_2026-10-11/` conserva README y capturas:

- https://github.com/sunsitox/sunsitox-Scraper_de_eventos_culturales/actions/runs/38127762874: run 25, commit bd7a052, éxito y dos artefactos según registro histórico y `screenshots/GitHub_scraper_run_25_success.png`.
- https://github.com/sunsitox/sunsitox-Scraper_de_eventos_culturales/actions/runs/35942705644: run 1, commit 0498e63, validación de fuentes según `screenshots/GitHub_validacion_fuentes_run_1_success.png`.
- `screenshots/GitHub_scraper_runs_scheduled.png`: antecedente de horarios; DEF-CI-001 permanece abierto respecto de inicio fuera de ventana, aunque el procesamiento terminase en éxito. El horario configurado no demuestra puntualidad.

No se volvió a consultar GitHub ni se descargaron sus artefactos en esta revisión. Estas referencias permiten auditar la afirmación histórica, no certificar ejecución actual.

8.3. Comandos de reproducción, no ejecutados en esta revisión

Los scripts frontend de package.json corresponden a ng test, ng build y ng lint:

```
# Desde EventoCultural_Integrado/EventoCultural_Integrado
npm test -- --watch=false
npm run build
npm run lint
# Desde tmp_scraper_repo, como declara scrape-events.yml
python -m unittest discover -s tests -v
```

Se mantienen guiones ASCII y texto monoespaciado. Cada futura ejecución debe registrar SHA o huellas de fuentes, entorno, fecha/zona, responsable, comando, resultado y artefactos sanitizados. Cantidad de pruebas no es cobertura; compilación no prueba RLS, OAuth, correo, moderación

ni Android. Los casos incrementales F3-01 a F3-10 y la revisión de 199 casos de la matriz no se convierten aquí en aprobados por existir implementación.

9. Despliegue sin Docker y brecha de aceptación

La estrategia documentada usa GitHub Actions para scraper, Vercel para SPA, Supabase administrado y Capacitor para Android. RNF-11 conserva un criterio que exige Dockerfile(s) y docker-compose para componentes aplicables. No se encontraron esos artefactos en las fuentes revisadas: hay una **brecha documental de aceptación**, no una excepción docente demostrada. No se añade Docker ni se cambia código funcional. Despliegue efectivo, configuración de correo, RLS/Storage y funcionamiento nativo requieren evidencias independientes del build.

10. Innovación y límites del MVP

La aportación es reunir fuentes heterogéneas en un catálogo con procedencia, normalización, deduplicación y vigencia; el scraper reutiliza checkpoints y enriquecimiento selectivo. Las recomendaciones del frontend se calculan por afinidad, no se describen como un LLM. No se cuantifican ahorro, latencia, usuarios o eventos vigentes sin medición fechada. El mapa puede usar ubicaciones aproximadas; no son coordenadas exactas certificadas. El MVP no acredita pagos ni monetización; propuestas comerciales quedan fuera de alcance.

11. Estándares de desarrollo y evidencia del avance

Esta sección declara los estándares de la industria bajo los que se desarrolla el proyecto, cómo se organiza la evidencia del avance bajo control de versiones y dónde pueden verificarse los aportes por integrante; responde así a los criterios de evaluación de "evidencias que cumplan estándares de desarrollo", "estructura y nombres" y "aporte por integrante".

11.1. Estándares, convenciones y herramientas

El cuadro distingue especificaciones externas, decisiones del equipo y herramientas. Ninguna de ellas acredita por sí sola conformidad o seguridad de producción.

Cuadro 3: Estándares, convenciones y herramientas con evidencia local.

Categoría	Elemento	Evidencia y límite
Estándar	WCAG 2.2	Formularios con `role="alert"` y `role="status"`; auditorías axe del 31-10 con hallazgos. No se declara conformidad AA.
Protocolo	OAuth 2.0	`src/app/core/services/supabase.service.ts`, iniciarOAuth y procesarCallbackOAuth. PKCE no acreditado; véase el apartado siguiente.
Mecanismo de BD	RLS PostgreSQL	Migraciones del scraper y aplicación; vista con security_invoker. Configuración remota no inspeccionada (Sección 5).
Convención	Estilo e inyección Angular	Componentes standalone, servicios e inject; revisión con `npm run lint`. ESLint no es auditoría de dependencias.

Categoría	Elemento	Evidencia y límite
Herramienta	Dependencias	package-lock.json fija resoluciones npm; requirements.txt declara dependencias Python. No se halló comando/workflow de npm audit, pip-audit, SCA o equivalente en las fuentes revisadas. No se certifica ausencia de vulnerabilidades.
Herramienta	CI del scraper	`.github/workflows/scrape-events.yml`: unittest antes de extracción y procesamiento; pasos de conservación con always(). Instalar dependencias no es auditarlas.
Configuración	Secretos	Funciones leen variables de entorno de servidor. No se inspeccionaron sus valores ni secretos de despliegue. Clave pública cliente no equivale a service_role.
Plataforma	Web y Android	angular.json, capacitor.config.ts y android/. Artefacto existente no demuestra funcionamiento nativo.

11.2. OAuth y alcance de la afirmación PKCE

En `src/app/core/services/supabase.service.ts:82-92`, createClient configura `detectSessionInUrl: false`, persistencia y refresco; **no configura flowType: 'pkce'**. La copia instalada de `node_modules/@supabase/auth-js/src/GoTrueClient.ts` contiene `flowType: 'implicit'` como valor predeterminado. El callback local distingue `?code=` de `#access_token`; la rama de código llama `exchangeCodeForSession(window.location.href)`, y la otra usa setSession. La documentación oficial de Supabase exige un código y el code verifier del inicio del flujo; la firma documentada es `exchangeCodeForSession(code)`. Por tanto, disponer de esa rama y de signInWithOAuth **no acredita PKCE configurado ni un canje correcto**. No se ejecutó un inicio OAuth completo, no se inspeccionó una solicitud con code_challenge ni el intercambio en red. Se retira la afirmación de cumplimiento de RFC 7636; validar configuración, argumento y recorrido queda pendiente, sin modificar código funcional.

11.3. Patrones de diseño y mecanismos observables

Las rutas del scraper de este cuadro son relativas a `tmp_scraper_repo/`; las del cliente, a la copia de la aplicación identificada en trazabilidad.

Patrón o mecanismo	Implementación comprobada y alcance
Strategy	`eventos/connectors/base.py`: Connector.collect(source); conectores concretos implementan la extracción intercambiable.
Factory / Registry	`eventos/connectors/registry.py`: register_connector y ConnectorFactory.create seleccionan la clase desde el registro por source.connector.
Adapter	En `src/app/core/services/`, archivo `supabase-recomendacion-persistencia.service.ts`: `SupabaseRecomendacionPersistencia` adapta el contrato de `recomendacion-persistencia.ts` a Supabase.
Facade	ApiService en `src/app/core/services/api.service.ts` delega operaciones de sesión en SupabaseService.

Patrón o mecanismo	Implementación comprobada y alcance
Caché condicional	ConditionalHttpCache en `eventos/cache.py`, usada por HttpClient en `eventos/http.py`. Se describe el mecanismo HTTP, no se afirma Cache-Aside genérico sin delimitarlo.
Concurrencia limitada	EventPipeline en `eventos/pipeline.py` usa ThreadPoolExecutor. Esto no basta para certificar el patrón Bulkhead ni aislamiento de todos los recursos.
Repository	SourceRepository en `eventos/config.py`: load() lee configuraciones JSON; EventPipeline recibe repository por constructor. Se limita al repositorio de fuentes, no a todo acceso a datos.
No acreditado	Se retira Chain of Responsibility: no se identificó una cadena de delegación entre manejadores. El procesamiento secuencial no la implica.

11.4. Roles planificados y aportes

Cuadro 5: Responsabilidades de planificación; no certificación de autoría.

Integrante	Responsabilidad planificada
Roberto Riffo	Liderazgo, frontend y UX.
José Miguel Moncada	Base de datos y QA.
David Herrera	Backend e integraciones.

Se conserva la planificación de la portada. No se atribuye la autoría de las funciones `reset-password-request` y `reset-password-confirm`, ni de otros cambios, sin commits o actas que la demuestren.

12. Trazabilidad de la revisión

Cuadro 6: Fuentes, versiones y alcance de la evidencia.

Componente	Identificación y límite
Frontend local	`EventoCultural_Integrado/EventoCultural_Integrado/`; package.json 0.0.1. No contiene metadatos Git; SHA, rama y diferencias contra un commit no recuperables. Huellas de fuentes en `Verificacion/fuentes.json`.
Referencia frontend	https://github.com/sunsitox/Proyecto_Capston_Evento_Cultural/tree/deploy/vercel-preview. Referencia declarada por el equipo, no equivalencia verificada ni inspección de su HEAD remoto.
Scraper local	https://github.com/sunsitox/sunsitox-Scraper_de_eventos_culturales; copia tmp_scraper_repo, rama local main, HEAD `bd7a05228ab7e2f5831f393eff20ae89e3872037`, fecha del commit 03-10-2026 10:27:11 -03:00. No se actualizó el remoto. git status registra modificación local de .env.example; no se usa su contenido como evidencia de producción.

Componente	Identificación y límite
Documento base	PDF y fuentes originales conservados en la carpeta superior; versión declarada 1.3 del 31-10-2026. Huellas previas en `Verificacion/originales.json`. Revisión 1.4 del 03-11-2026.
CI histórico	Run #25, `38127762874`, commit abreviado bd7a052; run #1 de validación de fuentes, `35942705644`, commit 0498e63. Registro y capturas consolidados el 11-10-2026 en la carpeta CI metodológica. No son ejecuciones de esta revisión.
QA histórico	`Evidencias_QA_Fase3_2026-10-31/controles.json` y logs: 25 frontend, 140 scraper, build/lint exit 0. No registran SHA ni huellas del código de esa ejecución: no se asegura identidad con las fuentes de hoy.
Revisión actual	Inspección estática, consulta de dos fuentes oficiales, renderizado de diagramas y compilación de PDF. No se ejecutaron pruebas de aplicación ni consultas SQL remotas.

Las carpetas de evidencia son hermanas de Documentacion Tecnica, bajo Documentacion Metodologica. Los enlaces permanentes de CI y los archivos concretos se detallan en la Sección 8. El número de pruebas ejecutadas no representa porcentaje de cobertura; ni cantidad de archivos ni líneas de pruebas acreditan aprobación.

13. Pendientes y evidencia faltante

ID	Punto pendiente	Evidencia esperada
P1	Esquema y autorización remotos	Exportación SQL o consultas de solo lectura: versión PostgreSQL, pg_get_viewdef, reloptions, propietarios, grants, RLS, pg_policies, historial de migraciones y pruebas anon/usuario/admin.
P2	OAuth / PKCE	Configuración efectiva del cliente, solicitud con code_challenge, verifier y canje de código sanitizados; sesión y cierre completos en web/Android.
P3	Versión frontend	Repositorio y SHA de la copia; git diff/status contra esa base, commit de entrega autorizado.
P4	RNF y regresión	Muestra/umbrales pendientes, mediciones de rendimiento, accesibilidad manual, dispositivos, aislamiento y ciclo borrador–moderación. Conservar criterios de la matriz; no inferir aprobación.
P5	CI y resultados históricos	Metadatos/artefactos originales y correspondencia entre SHA y fuentes probadas. Verificar vigencia remota y DEF-CI-001 sin reetiquetar historial.

ID	Punto pendiente	Evidencia esperada
P6	Despliegue reproducible	Acuerdo docente respecto de Docker exigido por RNF-11 o evidencias de su cumplimiento; reproducción limpia y configuración de servicios.
P7	Diccionario completo	Reconciliación de documentos históricos con exportación remota. Este documento incluye inventario y diccionario resumido local, no certifica cada columna remota.

Responsable y fecha objetivo de P1–P7: no constan en las fuentes revisadas. No se asignan arbitrariamente. La fecha 03-11-2026 corresponde a esta revisión, no a un compromiso de resolución. La ausencia de evidencia no demuestra una vulnerabilidad.

Referencias

Este documento cita los siguientes estándares y fuentes primarias (formato, estándar/autor, nombre, versión o RFC y URL de consulta):

[1] World Wide Web Consortium. *Web Content Accessibility Guidelines (WCAG) 2.2*. W3C Recommendation, oct. 2023. https://www.w3.org/TR/WCAG22/.

[2] Hardt, D. (ed.). *The OAuth 2.0 Authorization Framework*. RFC 6749, IETF, 2012. https://www.rfc-editor.org/rfc/rfc6749.

[3] Sakimura, N., Bradley, J. y Jones, M. *Proof Key for Code Exchange* (PKCE). RFC 7636, IETF, 2015. https://www.rfc-editor.org/rfc/rfc7636.

[4] PostgreSQL Global Development Group. *PostgreSQL Documentation: Row Security Policies*. https://www.postgresql.org/docs/current/ddl-rowsecurity.html.

[5] OpenStreetMap Foundation. *Nominatim Usage Policy* (geocodificación de respaldo del mapa). https://operations.osmfoundation.org/policies/nominatim/.

[6] Proyecto Mermaid. *Mermaid: documentación oficial* (herramienta utilizada para los diagramas propios 01–08). https://mermaid.js.org/intro/.

[7] Google LLC (equipo Angular). *Angular Style Guide* y documentación de componentes standalone. https://angular.dev/style-guide.

[8] Ionic. *Ionic Framework documentation* (v9) para navegación responsiva móvil/escritorio. https://ionicframework.com/docs.

[9] PostgreSQL Global Development Group. *CREATE VIEW*, documentación 18, apartado Notes, security_invoker y políticas del invocador. Consultado el 03-11-2026. https://www.postgresql.org/docs/current/sql-createview.html.

[10] Supabase. *PKCE flow*, configuración flowType, code verifier y canje de código. Consultado el 03-11-2026. https://supabase.com/docs/guides/auth/sessions/pkce-flow.

[11] Supabase. *Supabase documentation* (Row Level Security, Edge Functions y Inicio de sesión con GoTrue). https://supabase.com/docs.

[12] NVIDIA Corporation. *NVIDIA NIM for microservices* (API de inferencia de OCR y redacción). https://www.nvidia.com/en-us/ai-data-science/products/nim-microservices/.